



UNIVERSIDADE ESTÁCIO DE SÁ
PROGRAMAÇÃO FULL STACK
PÓLO NORTE SHOPPING

CLEYDSON ASSIS COELHO

MISSÃO PRÁTICA 1 MUNDO 3

RPG0014 - Iniciando o caminho pelo Java
Procedimento 1

RIO DE JANEIRO

2025

Objetivos da prática

1. Utilizar herança e polimorfismo na definição de entidades.
2. Utilizar persistência de objetos em arquivos binários.
3. Implementar uma interface cadastral em modo texto.
4. Utilizar o controle de exceções da plataforma Java.
5. No final do projeto, o aluno terá implementado um sistema cadastral em Java,
6. utilizando os recursos da programação orientada a objetos e a persistência em
7. arquivos binários.

JavaApplication.java

```
package javaapplication;
import model.*;
public class JavaApplication {
    public static void main(String[] args) {
        try {
            // Pessoas Físicas
            PessoaFisicaRepo repo1 = new PessoaFisicaRepo();
            repo1.inserir(new PessoaFisica(1, "Ana", "1111111111", 25));
            repo1.inserir(new PessoaFisica(2, "Carlos", "2222222222", 52));
            System.out.println("Dados de Pessoa Fisica Armazenados.");
            repo1.persistir("pessoas_fisicas.dat");

            PessoaFisicaRepo repo2 = new PessoaFisicaRepo();
            System.out.println("Dados de Pessoa Fisica Recuperados.");
            repo2.recuperar("pessoas_fisicas.dat");
            for (PessoaFisica pf : repo2.obterTodos()) {
                pf.exibir();
            }

            // Pessoas Jurídicas
            PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();
            repo3.inserir(new PessoaJuridica(3, "XPTO Sales", "3333333333333333"));
            repo3.inserir(new PessoaJuridica(4, "XPTO Solutions", "4444444444444444"));
            System.out.println("Dados de Pessoa Juridica Armazenados.");
            repo3.persistir("pessoas_juridicas.dat");

            PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();
            System.out.println("Dados de Pessoa Juridica Recuperados.");
            repo4.recuperar("pessoas_juridicas.dat");
            for (PessoaJuridica pj : repo4.obterTodos()) {
                pj.exibir();
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Pessoa.java

```
package model;
import java.io.Serializable;
public class Pessoa implements Serializable {
    private int id;
    private String nome;
    public Pessoa() {
    }
    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public void exibir() {
        System.out.println("Id: " + id);
        System.out.println("Nome: " + nome);
    }
}
```

PessoaFisica.java

```
package model;
import java.io.Serializable;
public class PessoaFisica extends Pessoa implements Serializable {
    private String cpf;
    private int idade;
    public PessoaFisica() {
        super();
    }
    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }
    public String getCpf() {
        return cpf;
    }
    public void setCpf(String cpf) {
        this.cpf = cpf;
    }
    public int getIdade() {
        return idade;
    }
    public void setIdade(int idade) {
        this.idade = idade;
    }
    @Override
    public void exibir() {
        System.out.println("Id: " + getId());
        System.out.println("Nome: " + getNome());
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }
}
```

PessoaFisicaRepo.java

```
package model;
import java.io.*;
import java.util.ArrayList;
public class PessoaFisicaRepo {
    private ArrayList<PessoaFisica> lista = new ArrayList<>();

    public void inserir(PessoaFisica pessoa) {
        lista.add(pessoa);
    }
    public void alterar(PessoaFisica pessoa) {
        for (int i = 0; i < lista.size(); i++) {
            if (lista.get(i).getId() == pessoa.getId()) {
                lista.set(i, pessoa);
                return;
            }
        }
    }
    public void excluir(int id) {
        lista.removeIf(p -> p.getId() == id);
    }
    public PessoaFisica obter(int id) {
        for (PessoaFisica p : lista) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }
    public ArrayList<PessoaFisica> obterTodos() {
        return lista;
    }
    public void persistir(String nomeArquivo) throws Exception {
        FileOutputStream fos = new FileOutputStream(nomeArquivo);
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(lista);
        oos.close();
        fos.close();
    }
    public void recuperar(String nomeArquivo) throws Exception {
        FileInputStream fis = new FileInputStream(nomeArquivo);
        ObjectInputStream ois = new ObjectInputStream(fis);
        lista = (ArrayList<PessoaFisica>) ois.readObject();
        ois.close();
        fis.close();
    }
}
```

PessoaJuridicaRepo.java

```
package model;
import java.io.*;
import java.util.ArrayList;
public class PessoaJuridicaRepo {
    private ArrayList<PessoaJuridica> lista = new ArrayList<>();

    public void inserir(PessoaJuridica pessoa) {
        lista.add(pessoa);
    }
    public void alterar(PessoaJuridica pessoa) {
        for (int i = 0; i < lista.size(); i++) {
            if (lista.get(i).getId() == pessoa.getId()) {
                lista.set(i, pessoa);
                return;
            }
        }
    }
    public void excluir(int id) {
        lista.removeIf(p -> p.getId() == id);
    }
    public PessoaJuridica obter(int id) {
        for (PessoaJuridica p : lista) {
            if (p.getId() == id) {
                return p;
            }
        }
        return null;
    }
    public ArrayList<PessoaJuridica> obterTodos() {
        return lista;
    }
    public void persistir(String nomeArquivo) throws Exception {
        FileOutputStream fos = new FileOutputStream(nomeArquivo);
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.writeObject(lista);
        oos.close();
        fos.close();
    }
    public void recuperar(String nomeArquivo) throws Exception {
        FileInputStream fis = new FileInputStream(nomeArquivo);
        ObjectInputStream ois = new ObjectInputStream(fis);
        lista = (ArrayList<PessoaJuridica>) ois.readObject();
        ois.close();
        fis.close();
    }
}
```

PessoaJuridica.java

```
package model;

import java.io.Serializable;

public class PessoaJuridica extends Pessoa implements Serializable {
    private String cnpj;

    public PessoaJuridica() {
        super();
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    public String getCnpj() {
        return cnpj;
    }

    public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        System.out.println("Id: " + getId());
        System.out.println("Nome: " + getNome());
        System.out.println("CNPJ: " + cnpj);
    }
}
```

Resultados:

Idade: 25

Id: 2

Nome: Carlos

CPF: 22222222222

Idade: 52

Dados de Pessoa Juridica Armazenados.

Dados de Pessoa Juridica Recuperados.

Id: 3

Nome: XPTO Sales

CNPJ: 33333333333333

Id: 4

Nome: XPTO Solutions

CNPJ: 44444444444444

BUILD SUCCESSFUL (total time: 0 seconds)

Análise e Conclusão:

1. Quais as vantagens e desvantagens do uso de herança?

Vantagens:

- Permite reaproveitar código, evitando duplicações.
- Facilita a manutenção e a organização do projeto em estruturas hierárquicas.
- Possibilita o uso do polimorfismo, tornando o sistema mais flexível.

Desvantagens:

- Pode causar acoplamento forte entre as classes, dificultando mudanças futuras.
- Quando mal aplicada, a herança pode deixar o código confuso e difícil de testar ou estender.
- Excesso de herança pode comprometer a simplicidade do projeto.

2. Por que a interface `Serializable` é necessária ao efetuar persistência em arquivos binários?

Ela é necessária porque diz ao Java que os objetos daquela classe podem ser convertidos em uma sequência de bytes. Isso é fundamental para que possamos salvar esses objetos em

arquivos binários (com `ObjectOutputStream`) e depois recuperá-los (`ObjectInputStream`) mantendo todos os dados e estruturas.

3. Como o paradigma funcional é utilizado pela API Stream no Java?

O paradigma funcional aparece na API Stream por meio de métodos como `map()`, `filter()`, `reduce()` e `forEach()`. Com esses métodos, conseguimos manipular coleções de dados de forma declarativa (sem laços explícitos), tornando o código mais limpo, conciso e eficiente.

4. Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?

O padrão mais comum é o da **serialização de objetos**, onde os dados são gravados e lidos por meio das classes `ObjectOutputStream` e `ObjectInputStream`. Esse padrão permite salvar o estado completo de objetos em arquivos binários e restaurá-los quando necessário, mantendo a integridade das informações.